
Database Connection Demo

— 100-of-100 team —

Cloud Database - AWS RDS

- Relational Database
- Easy to set up
- Have free-tier
- Portable across projects

- Make sure to set the database public if you need to connect locally or with other cloud platform

Choose a database creation method

☐ **Standard create**
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☒ **Easy create**
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Configuration

Engine type [Info](#)

☐ Aurora (MySQL Compatible)


☐ Aurora (PostgreSQL Compatible)


☒ MySQL


☐ MariaDB

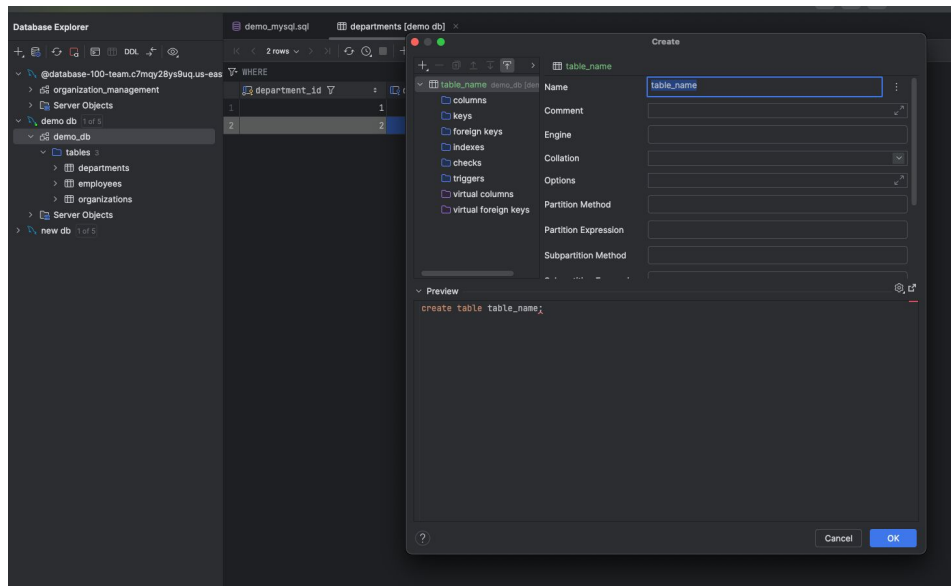

☐ PostgreSQL


☐ Oracle


☐ Microsoft SQL Server


DataGrip

- Free student license
- GUI
- Does not require much SQL knowledge
- Alternative - DBeaver



Project Configuration

- Repositories / pom.xml

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;
```

- Configurations

```
spring.datasource.url=jdbc:mysql://<RDS-endpoint>:3306/<database-name>  
spring.datasource.username=<username>  
spring.datasource.password=<password>
```

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-jpa</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>mysql</groupId>  
    <artifactId>mysql-connector-java</artifactId>  
    <scope>runtime</scope>  
  </dependency>  
</dependencies>
```

Code Demo

```
public class DatabaseDemo {  Tony Yu
    private static final String URL = "jdbc:mysql://demo-db.c3ugsummgbeu.us-east-1.rds.amazonaws.com:3306/demo_db"; 1 usage
    private static final String USER = "admin"; 1 usage
    private static final String PASSWORD = "12345678"; 1 usage

    public static void main(String[] args) {  Tony Yu
        Connection connection = null;
        try {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Connected to the database.");
        }
    }
}
```

Connection Setup

Query Example

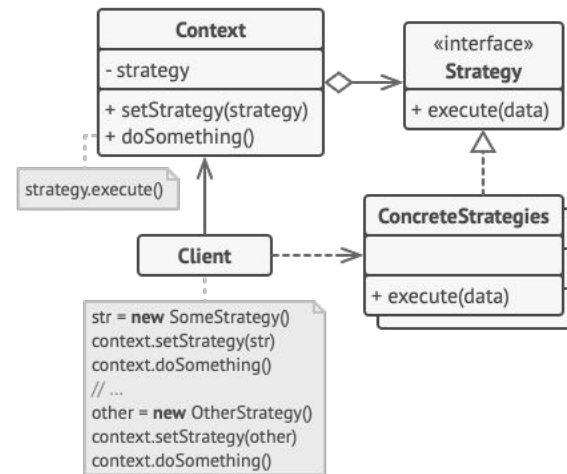
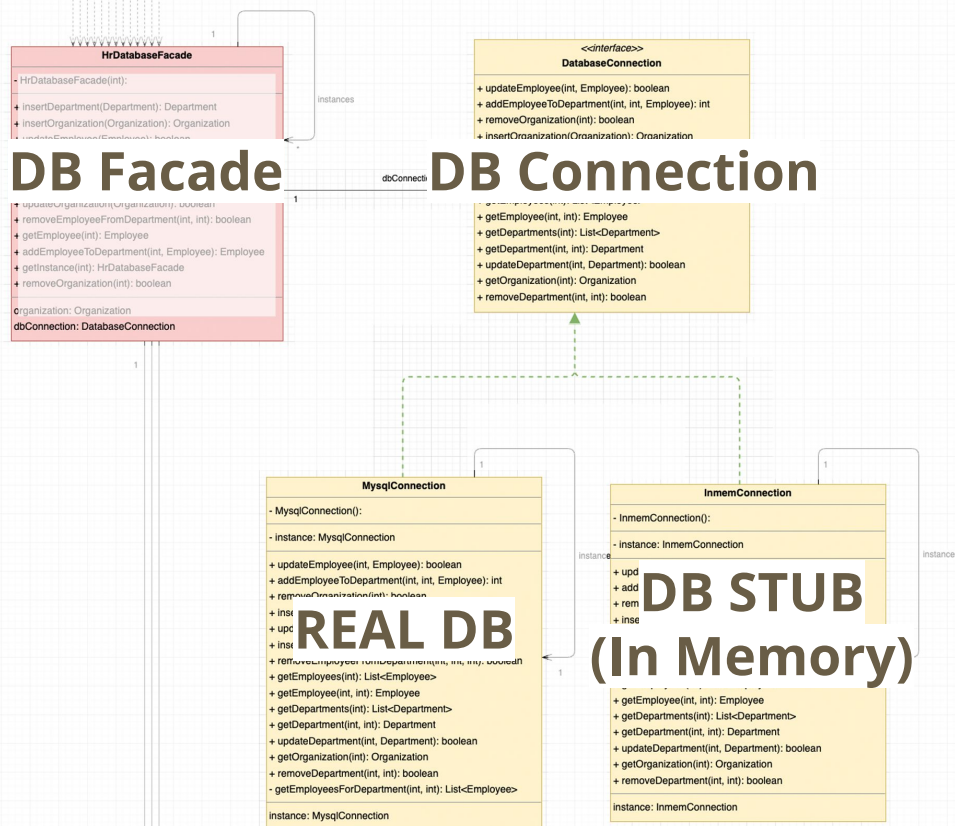
```
public static void getEmployee(Connection connection, int organizationId, int employeeId) {  Tony Yu
    String query = "SELECT * FROM employees WHERE organization_id = ? AND employee_id = ?";
    try (PreparedStatement pstmt = connection.prepareStatement(query)) {
        pstmt.setInt( parameterIndex: 1, organizationId);
        pstmt.setInt( parameterIndex: 2, employeeId);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            System.out.println("\nEmployee Details:");
            System.out.println("Employee ID: " + rs.getInt( columnLabel: "employee_id"));
            System.out.println("Name: " + rs.getString( columnLabel: "name"));
            System.out.println("Position: " + rs.getString( columnLabel: "position"));
            System.out.println("Hire Date: " + rs.getDate( columnLabel: "hire_date"));
            System.out.println("Salary: " + rs.getDouble( columnLabel: "salary"));
            System.out.println("Performance: " + rs.getDouble( columnLabel: "performance"));
        } else {
            System.out.println("\nEmployee not found.");
        }
    }
}
```

Extra Tips

- When we do unit testing, we use stub (fake) database to connect, instead of the real cloud database.
- We followed the Singleton and Strategy design pattern to manage our database connections.

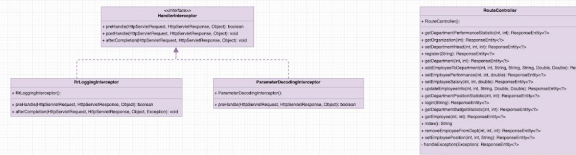
Architecture Design - Strategy Pattern



- Database Flexibility
- Open-Closed Principle (OCP)
- Isolation of Concerns

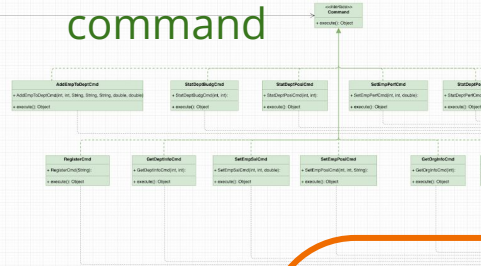
Any Questions?

Architecture Design

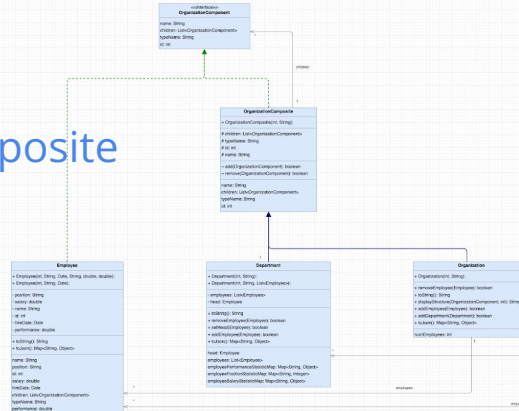


chain-of-responsibility

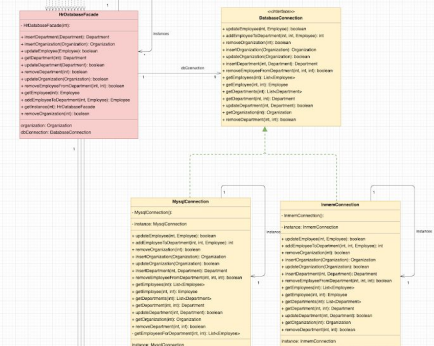
command



composite



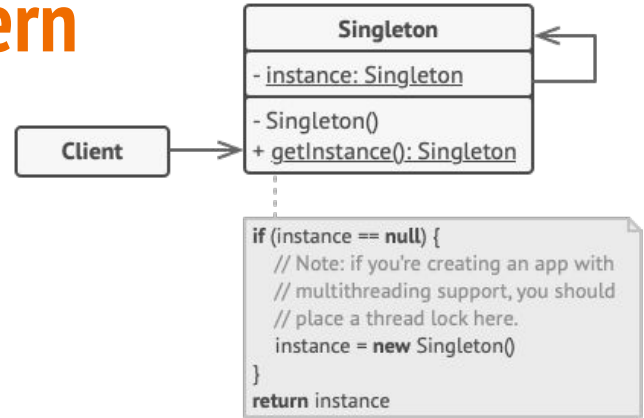
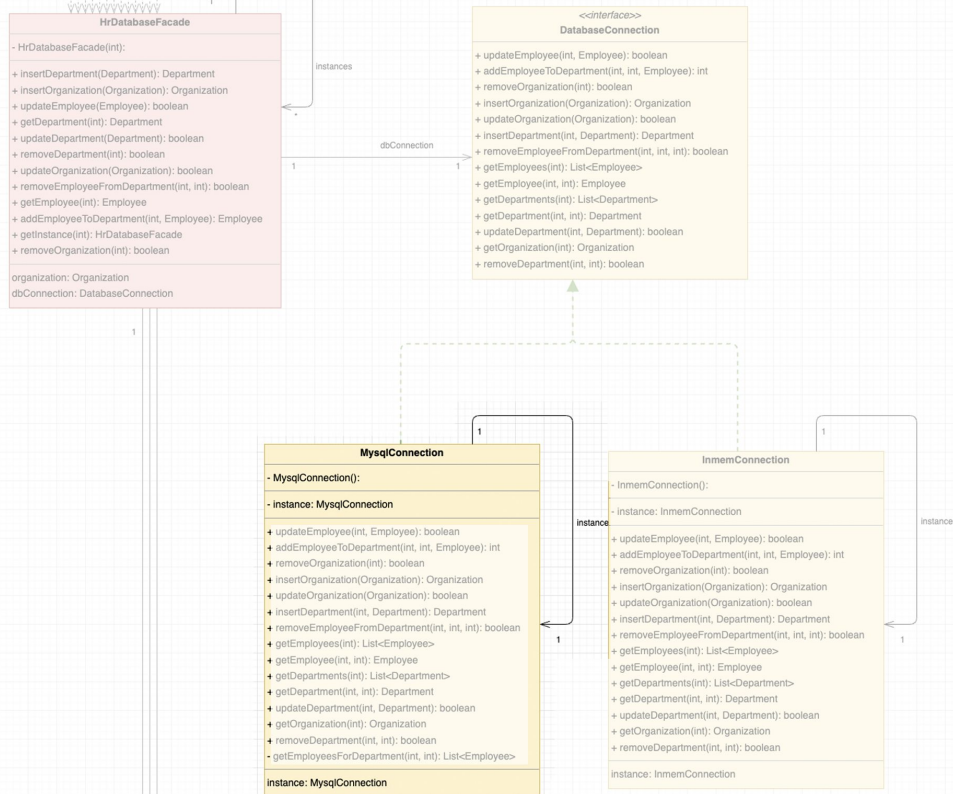
facade



strategy

singleton

Architecture Design - Singleton Pattern



- Resource Efficiency
- Concurrency Support
- Global Access Point